

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

TECHNICAL REPORT

MIDI Digital Synthesizer and Audio Effects Prototype

AUTHOR

Reza Sanaee Nezhad

Student ID: 401101502

INSTRUCTOR / INSTRUCTORS

Dr Mojahedian

ACADEMIC CONTEXT

Signals and Systems

Contents

1 Implementation scope 2

2 MIDI timing and envelopes 2

3 Effects and persistent state 2

4 Time-frequency processing and output 2

5 Module-by-module source inventory 3

6 Detailed oscillator and envelope design 4

7 Effect behavior and numerical boundaries 4

Python modules synthesize oscillator voices from MIDI notes, shape their amplitude envelopes, and process an audio-effects rack. The implementation combines note-frequency conversion, MIDI timing, modulation, filtering, nonlinear processing, and WAV export.

$$f(n) = 440\,2^{(n-69)/12}\ \text{Hz}.$$

1 Implementation scope

The code package includes waveform-generation scripts, individual effect modules, and finalCode.py. External MIDI paths referenced by scripts must be supplied for reproduction; each script should be configured with a local input file. The basic pipeline reads MIDI, maps note numbers to sinusoidal frequencies, applies an ADSR-style envelope, combines notes, processes effects, and exports a WAV waveform. The assignment's larger oscillator/effect wishlist should not be mistaken for every feature being robustly implemented.

2 MIDI timing and envelopes

MIDI delta times accumulate on a common timeline, and tempo determines their conversion to seconds. Each active note needs a start position and duration; simultaneous notes overlap rather than being concatenated. A note-on message with zero velocity conventionally terminates a note. Track-local timelines must be merged with tempo changes interpreted correctly. The timeline model accumulates absolute event time, pairs note-on/off events, creates a voice per note, and adds each voice into a common output buffer. Release extends after note-off rather than truncating abruptly. An envelope should multiply a voice once. The displayed envelope must agree with the waveform used for audio output. Envelope stage lengths must also fit short notes without negative durations or indexing errors.

3 Effects and persistent state

A delay requires a buffer whose contents persist across samples. The delay model places those initializations outside the loop and updates a circular buffer for each sample. Feedback must remain stable and fractional modulated delays require interpolation. Saturation should apply its drive/bias transformation to the signal before the nonlinear transfer function. The combined saturation routine uses x=drive*wave+bias as the input of its selected nonlinearity, then blends the wet and dry signals. A conventional compressor computes an envelope, compares its dB level to a threshold, applies a specified ratio, and smooths gain with attack/release; an ad hoc amplitude formula should be described as an approximation. Flanger and chorus use modulated delays on different time scales and voice arrangements. A phaser mixes all-pass-filtered audio with dry audio; accidentally adding dry components twice changes the gain and response. Reverberation approximates multiple decaying reflections rather than a single repeated echo. Each effect has its own transfer behavior and state requirements.

4 Time-frequency processing and output

Time-frequency arrays must have an explicit frequency-bin/time-frame convention. The pitch-shift module uses frequency bins along the first axis and time frames along the second. Correct phase-vocoder processing needs expected bin phase advance, phase unwrapping, a synthesis hop tied to the time-scale factor, and overlap-add normalization. WAV conversion must prevent out-of-range floating values from wrapping during int16 casting. Normalize with a nonzero-peak guard or clip

to the legal range before conversion, with gain policy documented. A correct effect implementation should be verified using impulses, sinusoids, silence, and bounded random input before relying on listening examples. Signal-processing checks use short deterministic stimuli; no listening-study result is claimed.

5 Module-by-module source inventory

The following entry points come from the supplied files. Module presence documents the prototype scope; it does not establish independent signal-quality validation.

5.1 chorus.py

`chorus_effect`.

5.2 combine.py

`generate_sine_wave`, `generate_sawtooth_wave`, `generate_triangle_wave`, `generate_square_wave`, `design_lowpass_filter`, `apply_filter`, `generate_lfo`, `generate_waveform_with_vcf_and_lfo`, `exponential_segment`, `generate_adsr_envelope`.

5.3 Compressor.py

`midi_to_freq`, `generate_sine_wave`, `generate_adsr_envelope`, `apply_compressor`, `midi_to_audio`.

5.4 delay.py

`delay_effect`.

5.5 finalCode.py

`midi_to_freq`, `generate_sine_wave`, `generate_adsr_envelope`, `midi_to_audio`, `butter_bandpass`, `bandpass_filter`, `zero_pad`, `envelope`, `vocoder`, `filter_bands`, `voice_encoder`, `delay_effect`, `hard_clipping`, `soft_clipping`, `variable_saturation`, `apply_saturation`, `apply_compression`, `phaser_effect`, `flinger_effect`, `chorus_effect`, `reverbration_effect`, `stft`, `istft`, `phase_vocoder`, `pitch_shift`, `apply_pitch_shift`.

5.6 flinger.py

`flinger_effect`.

5.7 generate_wave.py

`generate_sine_wave`, `generate_sawtooth_wave`, `generate_triangle_wave`, `generate_square_wave`, `design_lowpass_filter`, `apply_filter`, `generate_lfo`, `generate_waveform_with_vcf_and_lfo`.

5.8 generateWave.py

`generate_sine_wave`, `generate_sawtooth_wave`, `generate_triangle_wave`, `generate_square_wave`, `design_lowpass_filter`, `apply_filter`, `generate_lfo`, `generate_waveform`.

5.9 phaser.py

`phaser_effect`.

5.10 push.py

`exponential_segment`, `generate_adsr_envelope`.

5.11 reverbration.py

`reverbration_effect`.

5.12 saturation.py

`midi_to_freq`, `generate_sine_wave`, `generate_adsr_envelope`, `hard_clipping`, `soft_clipping`, `variable_saturation`, `apply_saturation`, `midi_to_audio_with_saturation`.

5.13 saturation2.py

`midi_to_freq`, `generate_sine_wave`, `generate_adsr_envelope`, `hard_clipping`, `soft_clipping`, `variable_saturation`, `apply_saturation`, `midi_to_audio_with_saturation`.

5.14 vocoder.py

`butter_bandpass`, `bandpass_filter`, `zero_pad`, `voice_encoder`.

6 Detailed oscillator and envelope design

The oscillator modules expose sine, sawtooth, square/triangle-style options where visible in source, plus filtering and low-frequency modulation. A sine oscillator uses sin(2*pi*f*t+phase), with duration converted to integer samples and endpoint excluded. Multiple detuned oscillators can create a richer tone, but also raise peak amplitude and require an explicit gain policy. A fixed filter design inside a routine is not automatically a sample-by-sample time-varying VCF. The combined prototype uses a nominal 44100-Hz sample rate and maps MIDI note n through 440*2^((n-69)/12). Stage durations become sample counts. An exponential segment should reach its stated endpoint continuously: a decay can use a normalized (1-exp(-alpha*t/T))/(1-exp(-alpha)) shape. Exponential segments are normalized to reach their specified endpoints; zero-duration stages return an empty segment. Zero-length stages and notes shorter than attack/decay require defined behavior. The assignment specifies an AHDSR hold stage; the submitted ADSR-named functions do not by themselves demonstrate an implemented hold stage.

7 Effect behavior and numerical boundaries

Delay and flanger depend on persistent circular-buffer state. Chorus mixes modulated delayed voices; reverberation combines decaying reflections. Saturation exposes hard/soft/variable clipping functions and drive/bias parameters. Compression estimates signal level and applies gain control. Vocoder modules expose filtering and time-frequency routines. Keep their order explicit because nonlinear effects do not commute. The module names flinger.py and reverbration.py are used by the supplied programs and imports. Useful checks are silence for NaN/division-by-zero behavior, an impulse for delay placement/decay, a sine for frequency/gain, and a short MIDI chord for simultaneous voices. These stimuli isolate numerical boundaries from subjective sound quality.